Appendix J
Configuration Commands

J.1 General Purpose Variable Definitions

There are 64 general-purpose integer variables, 256 general-purpose floating-point variables and 64 general-purpose status variables. These general-purpose variables exist and can be used by algorithms without variable definition. The purpose of the variable definition process is to attach variable names and initialization values to general-purpose variables.

Names have to be attached to variables, if they have to be displayed and if operator access is required. Initialization values will be used at start up time only, to set variables to their initial values.

- ***INT#:name [v=nnnn];**

  This defines integer variables. The use is totally open to the individual application.

#	Number of the integer to be defined (0 to 63). Undefined Integers may be used in algorithms, but will not be initialized and have no name attached.
name	Text string to be used as integer descriptor (maximum of 10 characters).
v	Initialization value set to nnnn (0 to ±9999).

  Example:

```
*INT23:BUFFIDX          v=132;    dead time buffer: float132 to 153
*FLOAT34:DEADTIME       v=0.3;    minutes
*FLOAT35:INPUT          v=50;
*FLOAT83:OUTPUT         v=50;
*ALGO5:DEADTIME         P1=83     P2=35     P3=34 P4=23 P5=24;
```

The parameters P4 and P5 of the DEADTIME algorithm have to be of integer type. Their purpose is to provide pointers for calculating history data. Parameter P4 contains the index number of the first of 22 sequential floating-point variables, to be used as dead time history buffer. Parameter P5 contains the index number of the floating-point variable, containing the oldest history value. No parameter definition for integer 24 is required, since integer 24 is used as an internal parameter of DEADTIME and neither initialization nor display is necessary.

- ***STATUS#:name [v=nnnn] [s0=string] [s2=string] ... [s7=string];**

This defines status variables. The use is totally open to the individual application.

#	Number of the status variable to be defined (0 to 63). Undefined status variables may be used in algorithms, but will not be initialized and have no name attached.
name	Text string to be used as variable descriptor (maximum of 10 characters).
v	Initialization value set to nnnn (0 to 7).
s0 to s7	Status descriptors to be used for display of up to 8 status strings depending on the value of the status variable (s0 for value of 0, s1 for value of 1 up to s6 for value of 6. s7 is the default for all other values). No regular space character is permitted within the descriptor and all status descriptors have to be of equal length. The character 'Ç' has to be used, wherever a SPACE character is needed. 'Ç' will be used like a SPACE character. The character 'Ç' is obtained by holding down the Alt-Key, while sequentially pressing the keys 1 2 8 on the numeric keypad.

Example:

```
*STATUS6:PVALARM      v=0  s0=ÇÇÇÇÇÇÇÇ  s1=INSTÇHIÇ  s2=INSTÇLOÇ
                           s3=ÇROCÇHIÇ   s4=ÇROCÇLOÇ
                           s5=ÇPVÇHIÇÇ
                           s6=ÇPVÇLOÇÇ   s7=ÇDEVÇHIÇ
                           s8=ÇDEVÇLOÇ;
*STATUS7:OPALARM1     v=0  s0=ÇÇÇÇÇÇÇÇ  s1=ÇOPÇHIÇÇ
                           s2=ÇOPÇLOÇÇ   s3=ÇINTÇHIÇ
                           s4=ÇINTÇLOÇ;
*INT1:STATUS1              v=0;
*ALGO5:ALARM               P1=6          P2=7 P3=1;
*DISPLAY2.UPDATE:stv6   col=6            row=17        fg=HI-RED
                           bg=BLACK;
*DISPLAY2.UPDATE:stv7   col=6            row=22        fg=HI-RED
                           bg=BLACK;
```

An integer variable is used as a general status variable in PID controllers as described in the chapter "Algorithms". In our example, integer 1, which has been given the name STATUS1, stores such controller status. Among other status information, integer 1 stores the total alarm status of the whole PID controller. To make status variables available for display configuration, the algorithm ALARM drives two status variables, status 6 (PVALARM1) and status 7 (OPALARM1).

- ***FLOAT#:name [v=nnnn];**

  This defines float variables. The use is totally open to the individual application.

#	Number of the float variable to be defined (0 to 255). Undefined float variables may be used, but will not be initialized and have neither name nor extended definitions attached.
name	Text string to be used as a float variable descriptor (maximum of 10 characters).
v	Initialization value set to nnnn (nnnn from 0.00 to ±9999.90).

  Example:

```
*FLOAT32:LAG-INPUT      v=20;
*FLOAT11:LAG-OUTPUT     v=20;
*FLOAT97:LAG-TC         v=0.5;
*ALGO20:LAG             P1=11     P2=32     P3=97;
```

  All three floating-point variables used by the LAG algorithm have been given names and start up values.

J.2 Extended Variable Definitions

Extended definitions for trend pens (Trend Display) can be made for up to 8 floating point variables. Similarly, extended definitions for general gar graphs can be made for up to 8 floating point variables. The floating-point variables extended for trend pens can be different from those extended for general bar graphs. However, there is no requirement that they have to be different.

- ***HISTORY#:fv## [tmin=nn][tmax=mm][c=Color];**

  This command assigns a float variable for history collection and activates history collection. **Display 4 is fixed for Trend**. Trend occupies text rows 1 to 14. Rows 0 and 15 to 24 are available for normal update.

#	Trend number (from 0 to 7) for which the command is valid.
fv##	Variable (float) to be assigned to history collection for trend #.
tmin=nn	Defines the bottom value (0% value) for trend and bar displays.
tmax=mm	Defines the top value (100% value) for trend and bar displays.
c=Color	Defines the Color for trend displays of the defined variable.

  Example:

```
*FLOAT32:LAG-INPUT      v=20;
*FLOAT11:LAG-OUTPUT     v=20;
*FLOAT97:LAG-TC         v=0.5;
*ALGO20:LAG             P1=11       P2=32       P3=97;
*HISTORY3:fv32          tmin=-20    tmax=260    c=CYAN
*HISTORY7:fv11          tmin=-20    tmax=260    c=CYAN
```

  All three floating-point variables used by the LAG algorithm have been defined first. Only input and output variables of the LAG algorithm have extended definitions for trending purposes. They have been assigned to trend pens 3 and 7. The trends will be displayed on display 3 (Trend Display) without further display configuration.

- ***LEVEL#:name [v=nnnn] [c1=cc] [r1=rr] [c2=cc] [r2=rr];**

  This defines level variables of float type. The use is totally open to the individual application. The display appears as a bar graph with the two diagonal corners c1-r1 and c2-r2.

  # Number of level variable to be defined (from 0 to 7).

  name Text string to be used as variable descriptor (maximum of 10 char).

v Initialization value set to nnnn (nnnn from 0.00 to +100.00). The range is fixed to 0.00=0% bar graph and 100.00=100% bar graph.

  c1 r1 c2 r2 Position parameters, representing the two diagonal corners of the level update area, within which a vertical bar graph update takes place.

  Example:

```
*FLOAT9:TANKLEVEL      v=50;
*LEVEL3:TANKLEVEL      v=50          c1=6  r1=5  c2=30  r2=15;
*DISPLAY1.UPDATE:lv3   fg=HI-BLUE  bg=BLACK;
*DISPLAY5.UPDATE:lv3   fg=HI-BLUE  bg=BLACK;
```

  The floating point variable 9 has been named TANKLEVEL and has been given extended definition as variable number 3 of type level with a defined area on screen. The display update commands DISPLAY1 and DISPLAY5 define the variable level 3 to be displayed in both, display 1 and 5.

J.3 Display Commands

All display commands, with the exception of command "***DISPLAY#.BACKGRND:filename;**", provide position and Color definitions. The existence of these options is shown in the command descriptions as follows:

- **..... [••••]**

  Format of these definitions:

  ..... [col=cc][row=rr][fg=colorname][bg=colorname]

  col=cc Char column on screen (cc from 0 to 39), where descriptor or update starts (left).

row=rr Char row on screen (rr from 0 to 23) of descriptor or update.

fg=colorname Foreground Color definition.

bg=colorname Background or character box Color definition.

colorname. The following names are available as Color definitions: BLACK (BK), BLUE (BL), GREEN (GN), CYAN (C), RED (R), MAGENTA (M), BROWN (BR), WHITE (W), GRAY (GY), HI-BLUE (HBL), HI-GREEN (HGN), HI-CYAN (HC), HI-RED (HR), HI-MAGENTA (HM), HI-YELLOW (HY), HI-WHITE (HW)

Example:

```
*FLOAT23:PRESSURE              v=45;
*DISPLAY1.UPDATE:fdesc23       col=12      row=5     fg=HI-WHITE
                               bg=BLUE;
*DISPLAY1.UPDATE:fv23          col=20      row=5     fg=HI-WHITE
                               bg=BLUE;
```

PRESSURE is the name of floating point variable number 23 and will be displayed in display 1, text row 5, starting at column 12. The actual value of floating point variable 23 will be displayed and updated continuously in display 1, text row 5, starting at text column 20.

- ***DISPLAY#.BACKGRND:filename;**

This command defines the file containing the background graphics data to be displayed.

Display number (0 to 7) for which the command is valid. If no DISPLAY#.BACKGRND command exists, the display is considered as non-existent by the system.

filename filename.EGA contains the background graphics data of display #. If filename.EGA does not exist, a black background will be used. See the chapter "Background Design" as well.

Example:

*DISPLAY6.BACKGRND:project;

If a graphics data file of the name PROJECT.EGA exists, the file contents will be used as background graphics for display 6. If NO graphics data file of the name PROJECT.EGA exists, the background of display 6 will be black. In both cases, display 6 is configured and useable (active).

- ***DISPLAY#.DESC:displayname, [••••];**

  # Display number (0 to 7) for which the command is valid.

  displayname Description of display #. The string must not have more than 30 characters and must ends with a ',' as a parameter separator. The ',' is not part of the descriptor any more. This way, SPACE characters are permitted within the descriptor itself and are not considered as end of descriptor.

  Example:

  *DISPLAY3.DESC:Tuning Trend,col=3 row=24
  fg=HI-GREEN bg=BLACK;

  Display 3 will display the title "Tuning Trend" in green on black background, in text row 24, starting at text column 3.

- ***DISPLAY#.UPDATE:fv## [••••];**

  This defines a float variable to be updated at a position of column and row.

  # Display number (0 to 7) for which the command is valid.

  fv## Float variable to be displayed and continuously updated.

- ***DISPLAY#.UPDATE:fdesc## [••••];**

  This defines a float variable descriptor to be displayed at a position of column and row.

#	Display number (0 to 7) for which the command is valid.
fdesc##	Float variable descriptor to be displayed. Update of descriptor takes place at display call-up only.

  Example:

```
*FLOAT23:PRESSURE          v=45;
*DISPLAY1.UPDATE:fdesc23   col=12   row=5   fg=HI-WHITE   bg=BLUE;
*DISPLAY1.UPDATE:fv23      col=20   row=5   fg=HI-WHITE   bg=BLUE;
```

  PRESSURE is the name of floating point variable 23 and will be displayed in display 1, text row 5, starting at column 12. The actual value of floating point variable 23 will be displayed and updated continuously in display 1, text row 5, starting at text column 20.

- ***DISPLAY#.UPDATE:iv## [••••];**

  This defines an integer variable to be updated at a position of column and row.

#	Display number (0 to 7) for which the command is valid.
iv##	Integer variable to be displayed and continuously updated.

- ***DISPLAY#.UPDATE:idesc## [••••];**

  This defines an integer variable descriptor to be displayed at a position of column and row.

#	Display number (0 to 7) for which the command is valid.
idesc##	Integer variable descriptor to be displayed. Update of descriptor takes place at display call-up only.

Example:

```
*INT2:COUNTER              v=45;
*DISPLAY7.UPDATE:idesc2    col=12  row=5     fg=HI-WHITE     bg=BLUE;
*DISPLAY7.UPDATE:iv2       col=20  row=5     fg=HI-WHITE     bg=BLUE;
```

COUNTER is the name of integer variable 2 and will be displayed in display 7, text row 5, starting at column 12. The actual value of integer variable 2 will be displayed and updated continuously in display 7, text row 5, starting at text column 20.

- ***DISPLAY#.UPDATE:stv## [••••];**

  This defines a status variable to be displayed and continuously updated at a position of column and row.

#	Display number (0 to 7) for which the command is valid.
stv##	Status variable to be displayed and continuously updated.

- ***DISPLAY#.UPDATE:stdesc## [••••];**

  This defines a status variable descriptor to be displayed at a position of column and row.

#	Display number (0 to 7) for which the command is valid.
stdesc##	Status variable descriptor to be displayed. Update of descriptor takes place at display call-up only.

Example:

```
*STATUS5:ALARM             v=1;
*DISPLAY6.UPDATE:stdesc5   col=12    row=5     fg=HI-WHITE    bg=BLUE;
*DISPLAY6.UPDATE:stv5      col=20    row=5     fg=HI-WHITE    bg=BLUE;
```

ALARM is the name of status variable 5 and will be displayed in display 6, text row 5, starting at column 12. The actual status of status variable 5 will be displayed and updated continuously in display 6, text row 5, starting at text column 20.

- ***DISPLAY#.UPDATE:lv## [••••];**

  This defines a level variable to be displayed and continuously updated. The position of column and row in this command will be ignored, since the definition of the "*LEVEL#" command already defines the position. The position of a level variable is the same for each display it is configured.

#	Display number (0 to 7) for which the command is valid.
lv##	Level variable to be displayed and updated.

- ***DISPLAY#.UPDATE:ldesc## [••••];**

  This defines a level variable descriptor to be displayed at a position of column and row.

#	Display number (0 to 7) for which the command is valid.
idesc##	Integer variable descriptor to be displayed. Update of descriptor takes place at display call-up only.

  Example:

```
*FLOAT9:TANKLEVEL          v=50;
*LEVEL3:TANKLEVEL          v=50      c1=6      r1=5      c2=30     r2=15;
*DISPLAY1.UPDATE:lv3       fg=HI-BLUE          bg=BLACK;
*DISPLAY5.UPDATE:lv3       fg=HI-BLUE          bg=BLACK;
*DISPLAY5.UPDATE:ldesc3    col=6     row=17    fg=HI-GREEN         bg=BLUE
```

  The floating point variable 9 has been named TANKLEVEL and has been given extended definition as variable number 3 of type level with a defined area on screen. The display update commands DISPLAY1 and DISPLAY5 define the variable level 3 to be displayed in both, display 1 and 5. The descriptor of variable level 3 is displayed in display 5 only.

- ***DISPLAY#.BAR:## [••••];**

  Only those parameters which were assigned for history and trend can be displayed as bar graphs. Bar graphs have a height of 100 pixels and a width of 3 pixels. The update overwrites text, overlapping the bar update area. The bar update area is one character wide and 8 characters high (rows 1 to 8).

  # Display number (0 to 7) for which the command is valid.

  ## History (Trend) number (0 to 7) for which the command is valid.

  Note: Row definition is fixed. Any row definition will be ignored.

  Example:

```
*FLOAT11:PV              v=250;
*HISTORY7:fv11  tmin=0 tmax=500        c=CYAN
*DISPLAY2.BAR:7          col=6    fg=HI-CYAN    bg=BLACK;
```

  Floating point variable 11 has been designed as trend pen 7. The initialization value of 250 represents 50% of range for trend and bar graph as well. Bar graph update takes place in display 2 column 6.

- ***DISPLAY#.MARKER:## [••••];**

  Only those parameters which were assigned for history and trend can be displayed as markers. Markers occupy a height of 100 pixels and a width of 3 pixels. The update overwrites text, overlapping the marker update area. The marker update area is one character wide and 8 characters high (rows 1 to 8).

  # Display number (0 to 7) for which the command is valid.

  ## History (Trend) number (0 to 7) for which the command is valid.

  Note: Row definition is fixed. Any row definition will be ignored.

Example:

```
*FLOAT10:SP              v=250;
*HISTORY7:fv11           tmin=0      tmax=500      c=CYAN
*DISPLAY2.MARKER:7       col=4       fg=HI-CYAN    bg=BLACK;
```

Floating point variable 11 has been designed as trend pen 7. The initialization value of 250 represents 50% of range for trend and marker as well. Marker update takes place in display 2 column 4.

J.4 Algorithm Definitions

Algorithm definitions are used to define the algorithms their sequence of execution. The algorithms make use of general purpose variables as explained in the chapter *"Algorithms"*.

- ***ALGO#:algoname P1=vnbr P2=vnbr P3=vnbr ... Pn=vnbr;**

  The ALGO command defines the algorithm to be calculated. There is no direct association between any algorithm and tags or displays. It is the task of the person configuring the system to keep the algorithms organized properly. The sequence of calculations within each scan is determined by the digits which make up the last characters of the ALGO# command. (Example: ALGO5 is the algorithm calculated fifth).

#	Sequence number defining when the algorithm is calculated in sequence. The maximum number of algorithms for the whole system is 128. Therefore '#' can be from 0 to 127.
algoname	Has to be a legal and proper name of an algorithm supported. A list of supported algorithms is in a separate chapter "Algorithms".
P1 to Pn	The number of parameters is dependent on the type of algorithm chosen. vnbr defines which variable has to be passed on to the algorithm. There is no need to specify whether it is an integer or float variable, the algorithm picks the right one.

Note: Be careful and know, whether the algorithm is using an integer, an integer with status strings or a float variable.